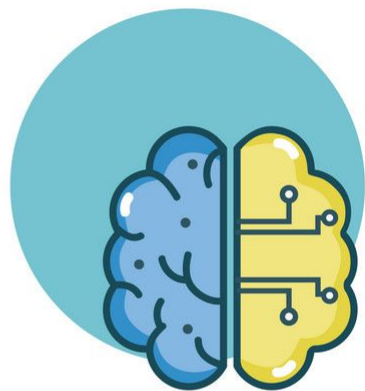


# INTRODUCTION TO MACHINE LEARNING

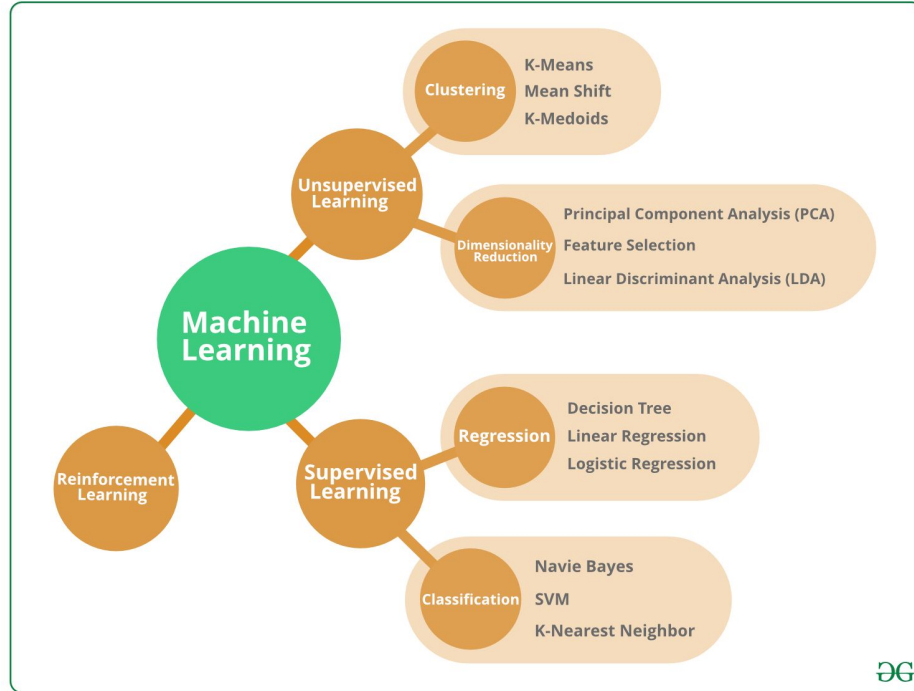
## LINEAR MODELS



Elisa Ricci



# MACHINE LEARNING MODELS



# MACHINE LEARNING MODELS

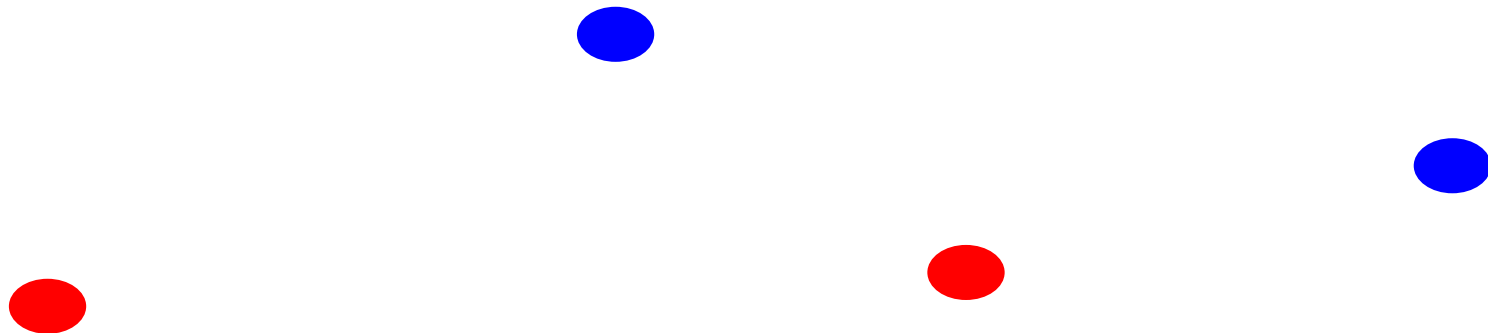
Some machine learning approaches make strong assumptions about the data

- If the assumptions are true it can often lead to better performance
- If the assumptions aren't true, the approach can fail miserably

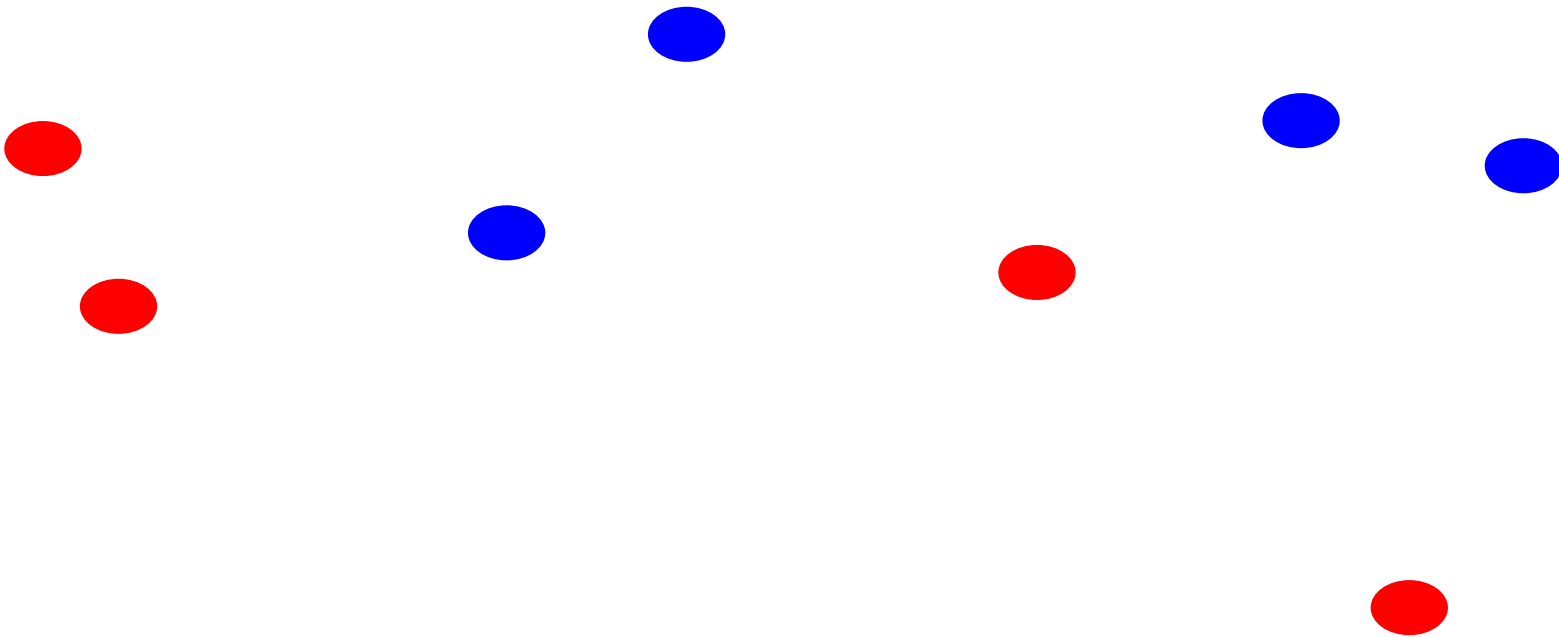
Other approaches don't make many assumptions about the data

- This can allow us to learn from more varied data
- But, they are more prone to overfitting and generally require more training data

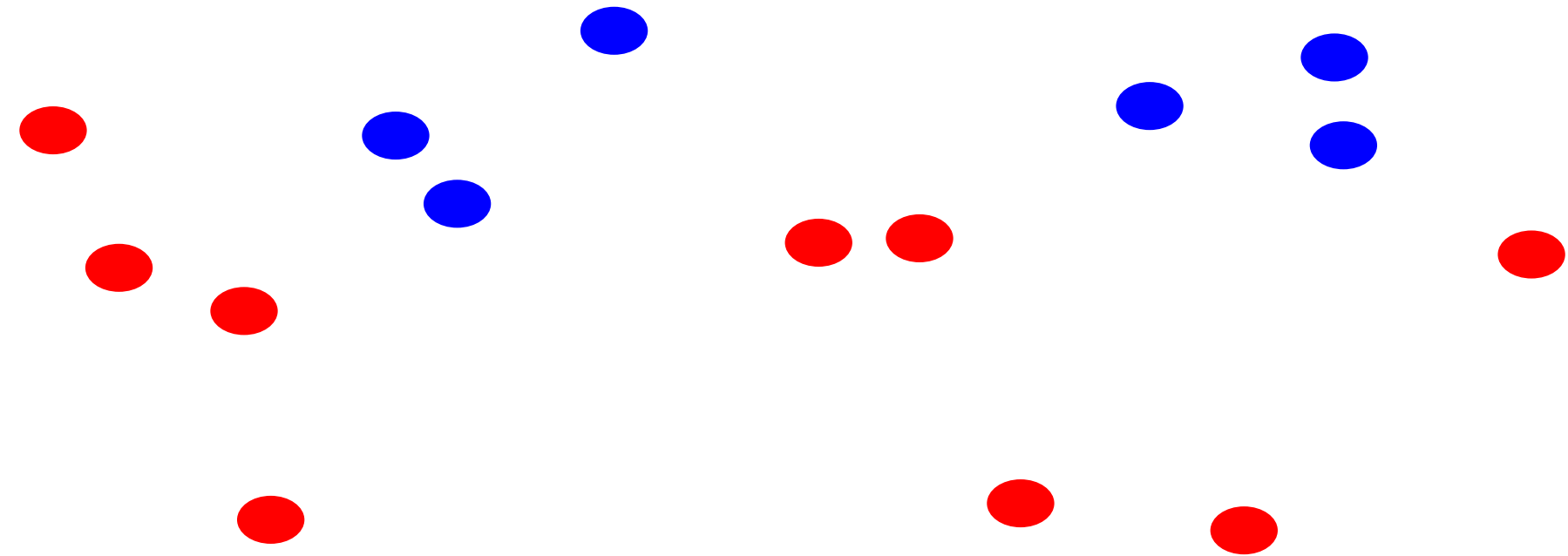
# WHAT IS THE DATA GENERATING DISTRIBUTION?



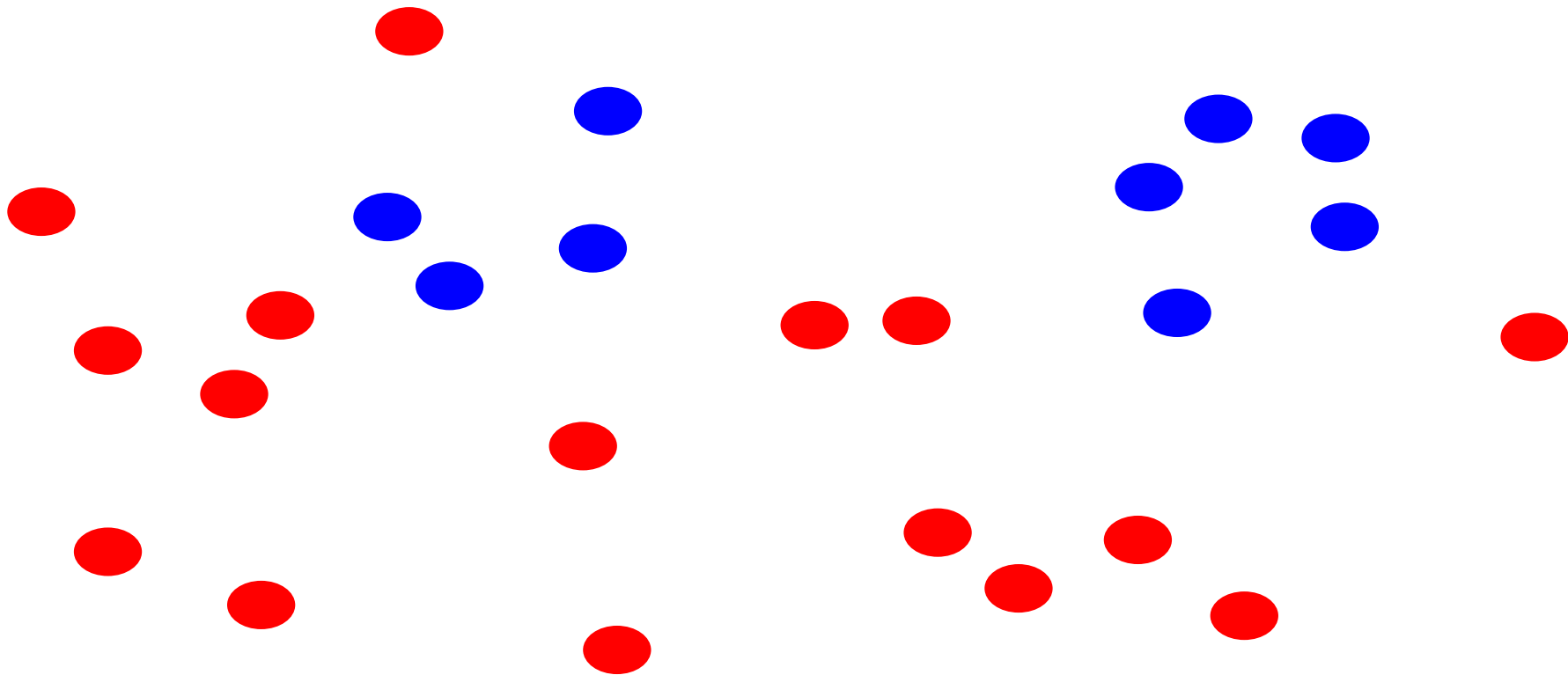
# WHAT IS THE DATA GENERATING DISTRIBUTION?



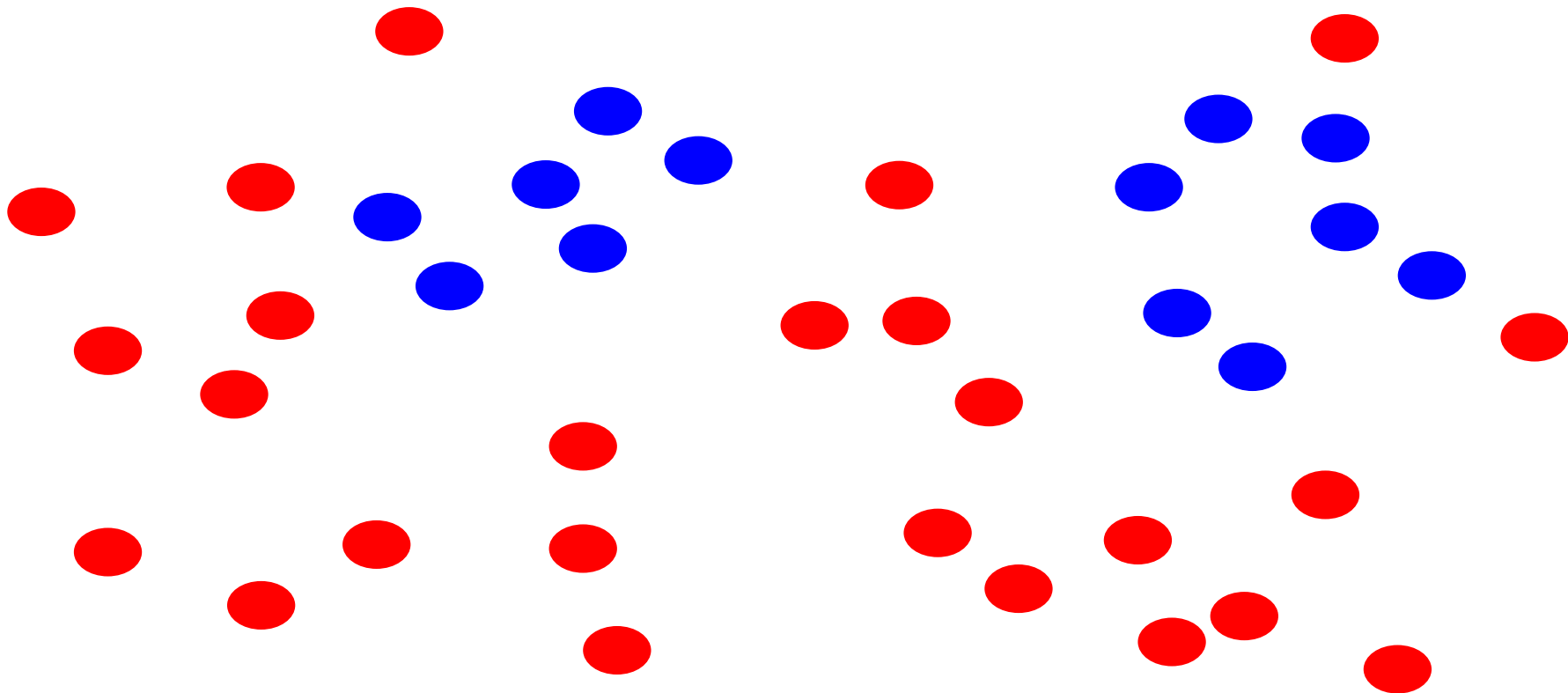
# WHAT IS THE DATA GENERATING DISTRIBUTION?



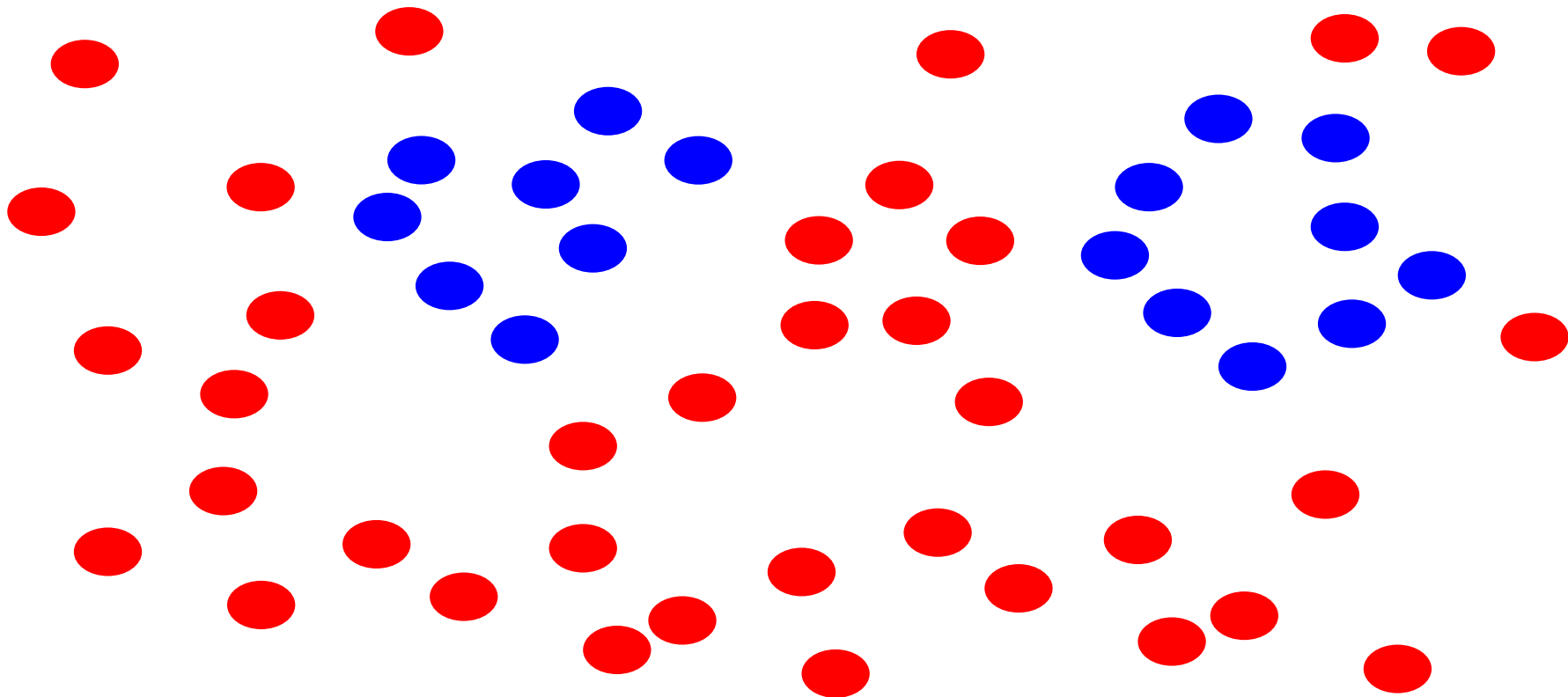
# WHAT IS THE DATA GENERATING DISTRIBUTION?



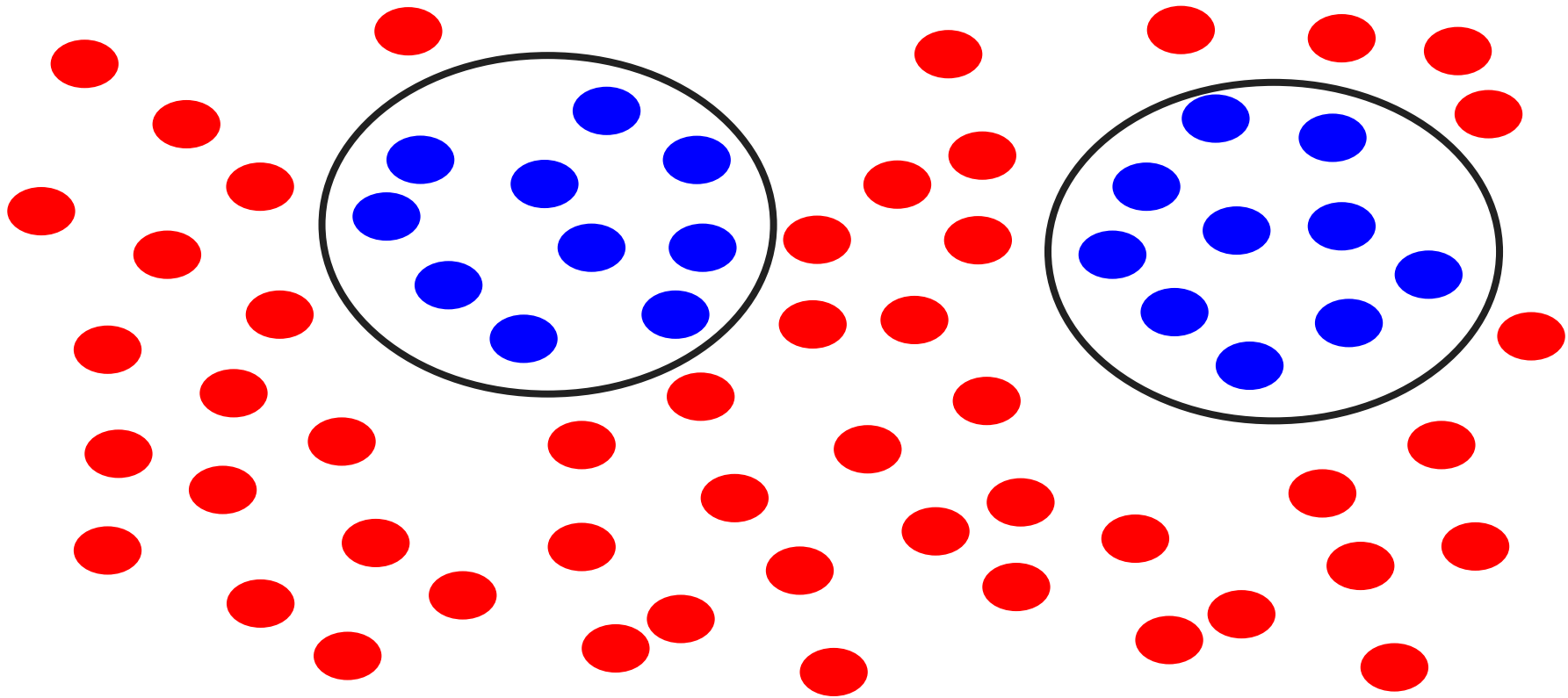
# WHAT IS THE DATA GENERATING DISTRIBUTION?



# WHAT IS THE DATA GENERATING DISTRIBUTION?



ACTUAL MODEL

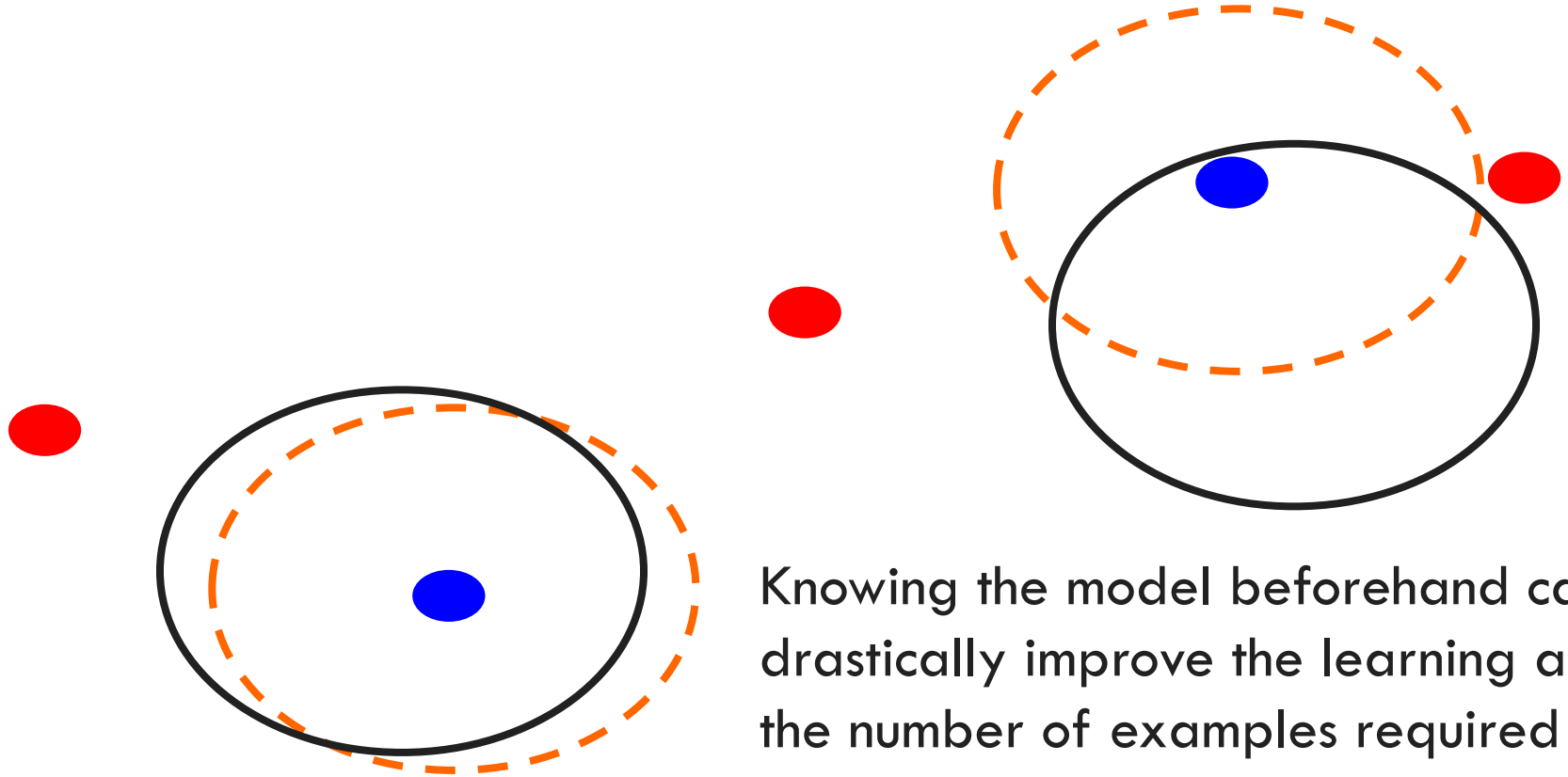


# MODEL ASSUMPTIONS

If you don't have strong assumptions about the model, it can take you a longer to learn

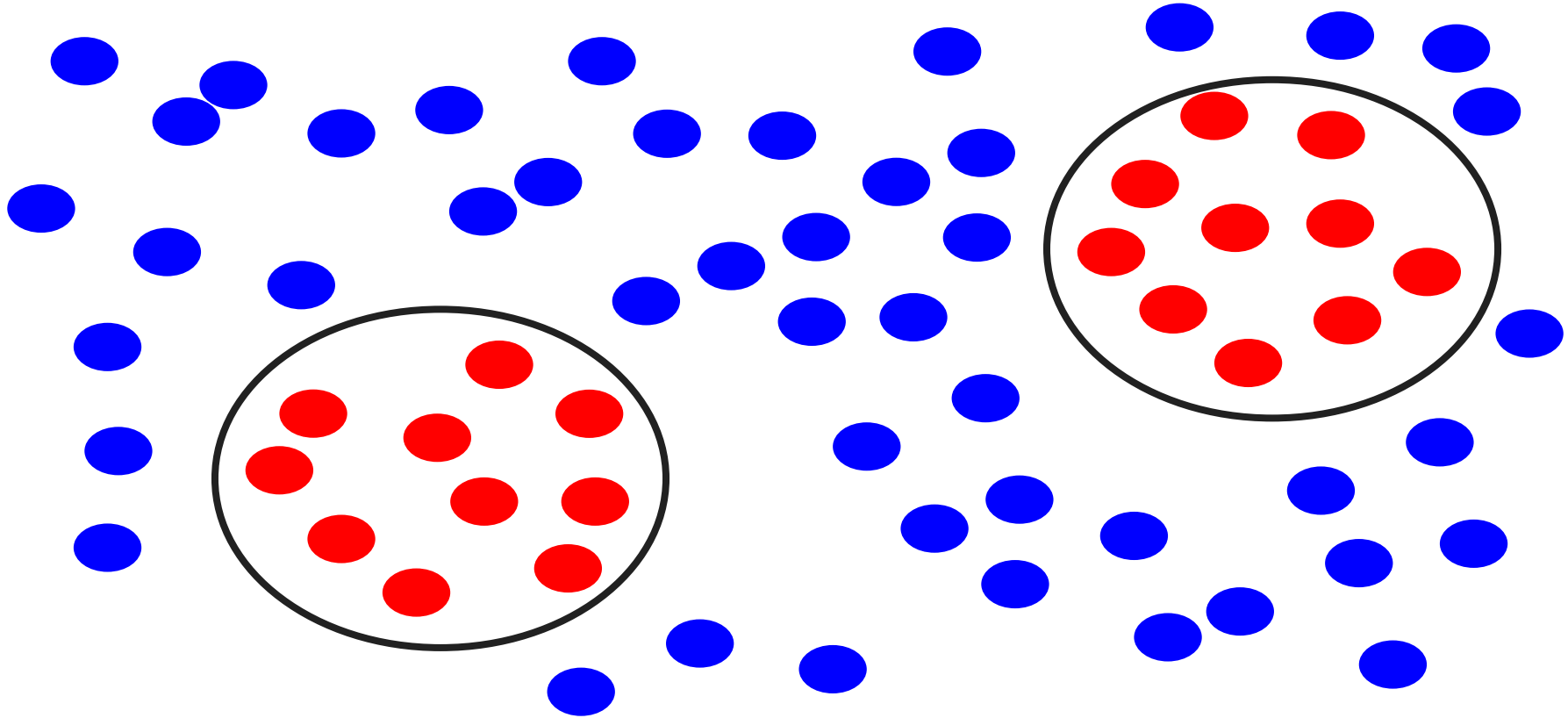
Assume now that our model of the blue class is two circles

# WHAT IS THE DATA GENERATING DISTRIBUTION?



Knowing the model beforehand can drastically improve the learning and the number of examples required

MAKE SURE YOUR ASSUMPTION IS CORRECT, THOUGH!

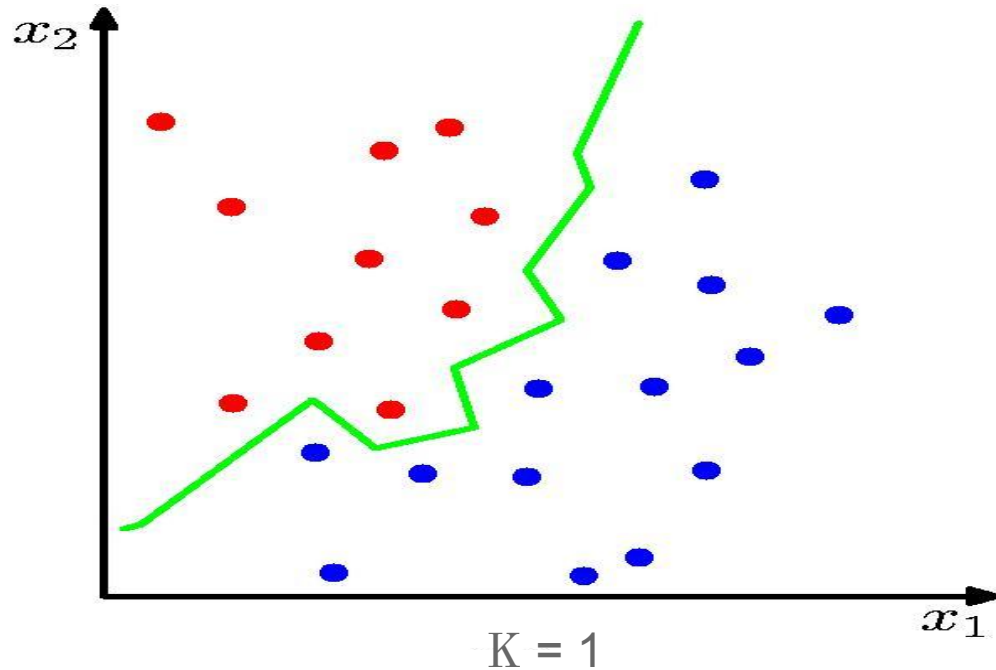


# MACHINE LEARNING MODELS

What are the **model assumptions** (if any) that  $k$ -NN make about the data?

Are there data sets that could never be learned correctly by it?

# K-NEAREST NEIGHBOR (K-NN)



No model assumptions. Assumes that proximity relates to class  
kNN can learn any arbitrary separation between the classes

# BIAS

The “**bias**” of a model is how strong the model assumptions are.

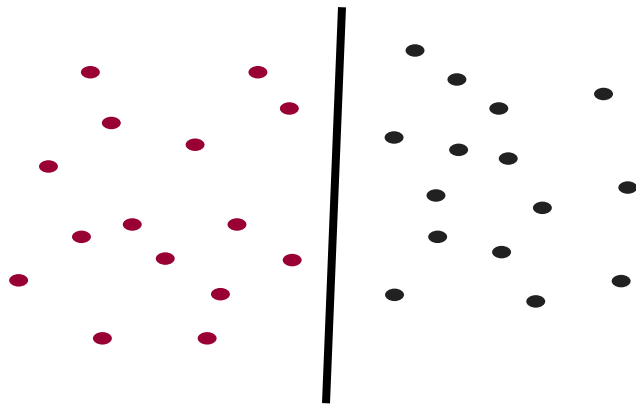
- low-bias classifiers make minimal assumptions about the data ( $k$ -NN and decision trees are generally considered low bias)
- high-bias classifiers make strong assumptions about the data

# LINEAR MODELS

A strong high-bias assumption is **linear separability**:

- in 2 dimensions, can separate classes by a line
- in higher dimensions, need hyperplanes

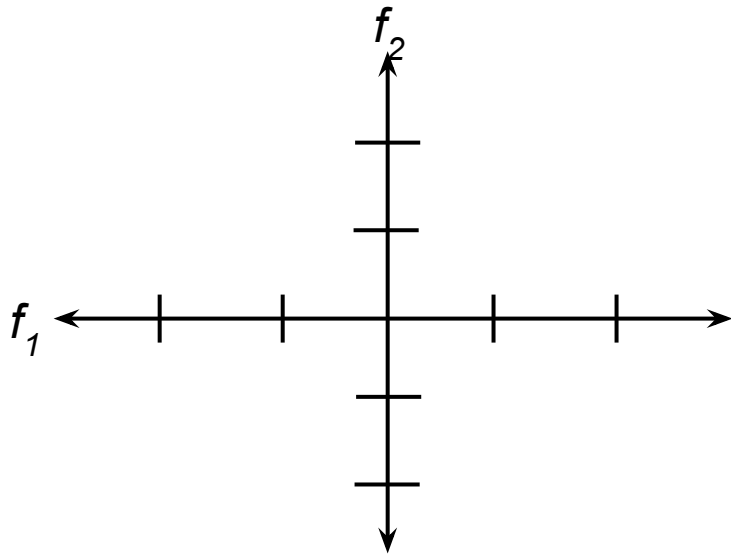
A **linear model** is a model that assumes the data is linearly separable



# DEFINING A LINE

Any pair of values  $(w_1, w_2)$  defines a line through the origin:

$$0 = w_1 f_1 + w_2 f_2$$

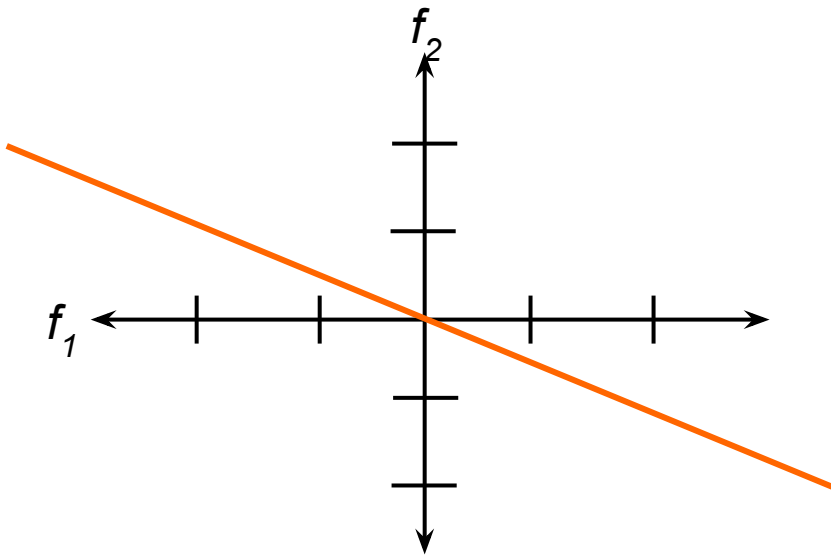


# DEFINING A LINE

$$0 = w_1 f_1 + w_2 f_2$$

$$0 = 1f_1 + 2f_2$$

|           |             |
|-----------|-------------|
| <b>-2</b> | <b>1</b>    |
| <b>-1</b> | <b>0.5</b>  |
| <b>0</b>  | <b>0</b>    |
| <b>1</b>  | <b>-0.5</b> |
| <b>2</b>  | <b>-1</b>   |



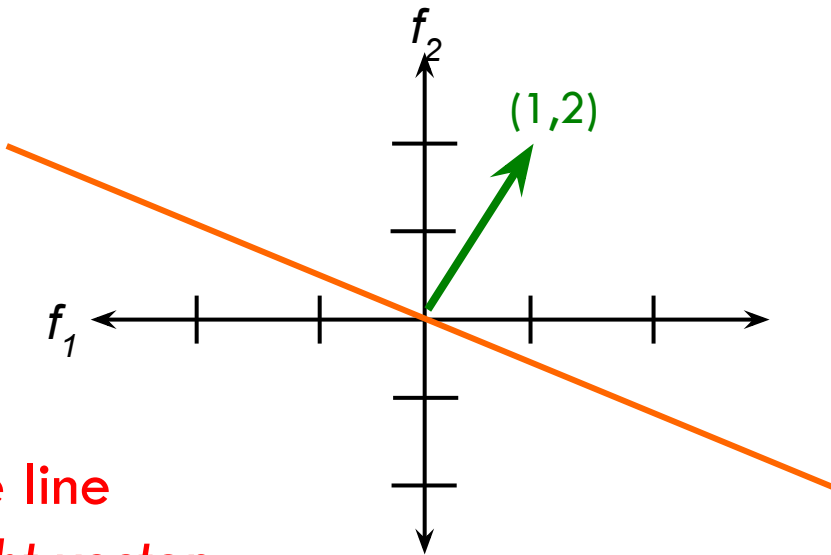
# DEFINING A LINE

$$0 = w_1 f_1 + w_2 f_2$$

$$0 = 1f_1 + 2f_2$$

$$\mathbf{w} = (1, 2)$$

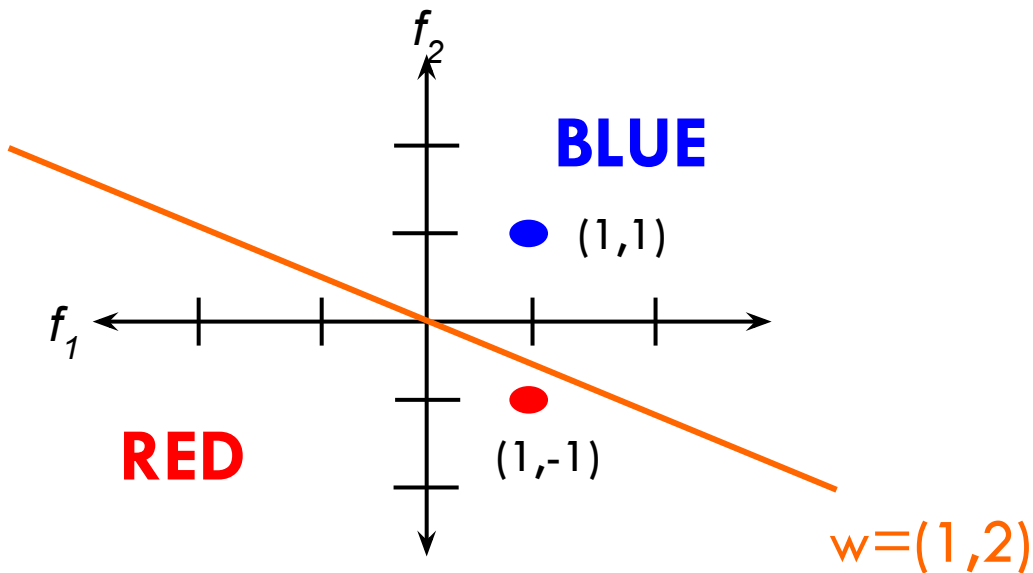
We can also view it as the line  
perpendicular to the *weight* vector



# CLASSIFYING WITH A LINE

Mathematically, how can we classify points based on a line?

$$0 = 1f_1 + 2f_2$$

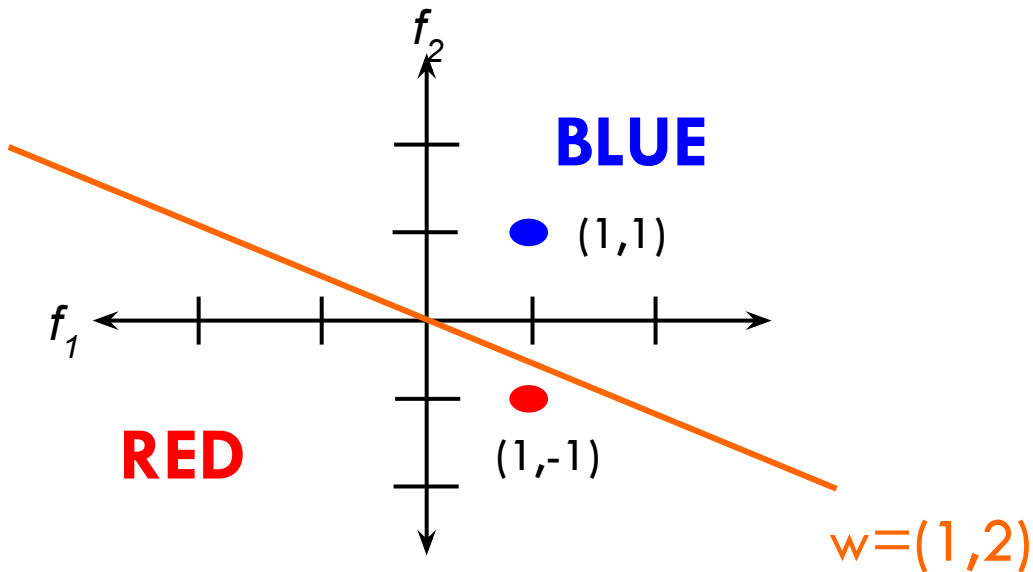


# CLASSIFYING WITH A LINE

$$0 = 1f_1 + 2f_2$$

$$(1,1): 1*1 + 2*1 = 3$$

$$(1,-1): 1*1 + 2*(-1) = -1$$

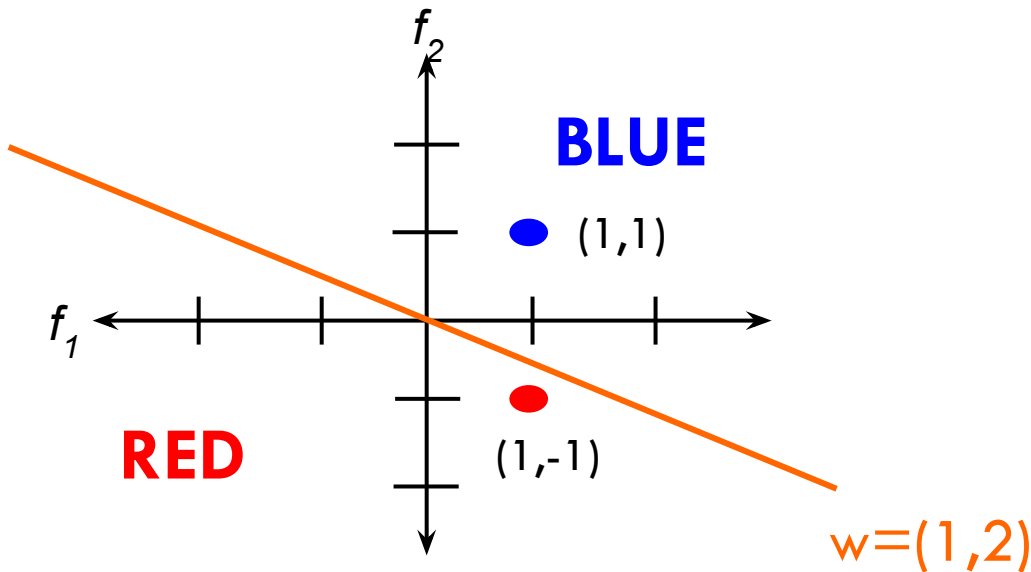


# CLASSIFYING WITH A LINE

$$0 = 1f_1 + 2f_2$$

$$(1,1): 1*1 + 2*1 = 3$$

$$(1,-1): 1*1 + 2*(-1) = -1$$



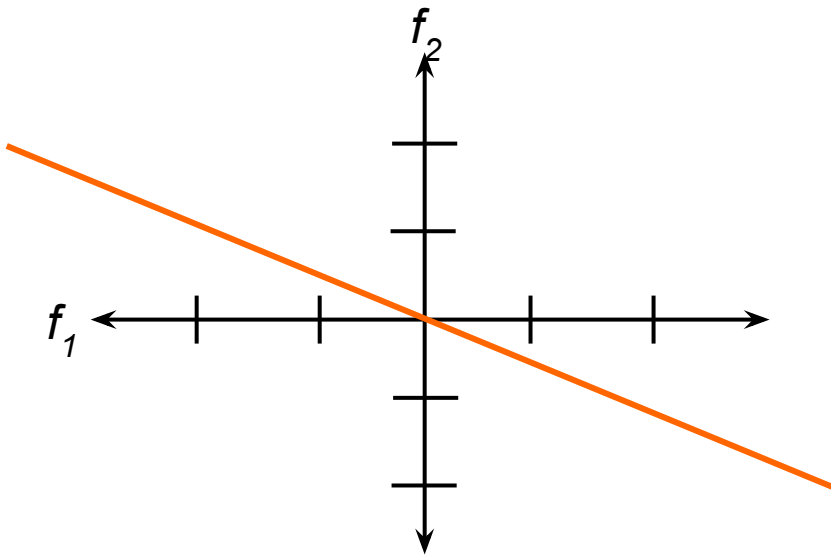
The sign indicates which side of the line

# DEFINING A LINE

Any pair of values  $(w_1, w_2)$  defines a line through the origin:

$$0 = w_1 f_1 + w_2 f_2$$

$$0 = 1f_1 + 2f_2$$



How do we move the line off of the origin?

# DEFINING A LINE

Any pair of values  $(w_1, w_2)$  defines a line through the origin:

$$a = w_1 f_1 + w_2 f_2$$

$$-1 = 1f_1 + 2f_2$$

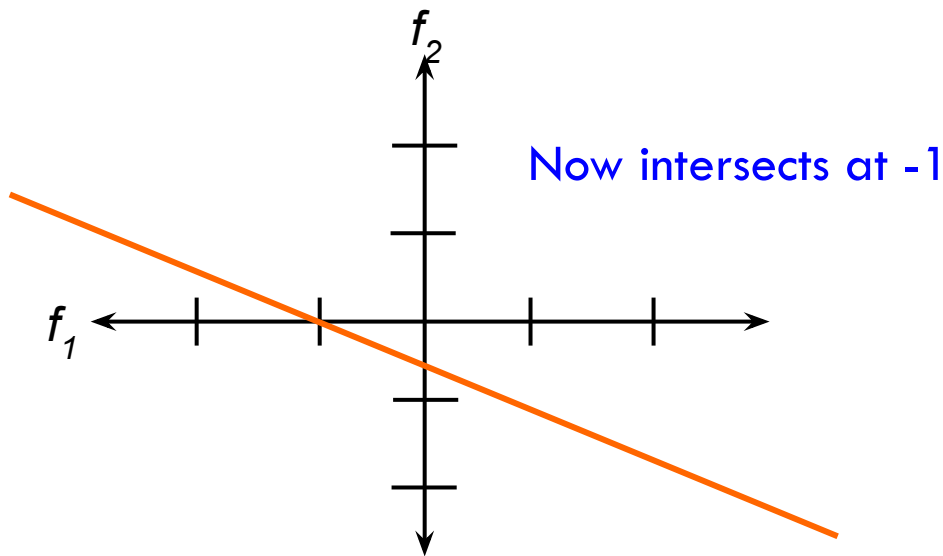
$$\mathbf{-2} \quad \mathbf{0.5}$$

$$\mathbf{-1} \quad \mathbf{0}$$

$$\mathbf{0} \quad \mathbf{-0.5}$$

$$\mathbf{1} \quad \mathbf{-1}$$

$$\mathbf{2} \quad \mathbf{-1.5}$$



# LINEAR MODELS

A linear model in  $n$ -dimensional space (i.e.  $n$  features) is defined by  $n+1$  weights. In two dimensions, we have a line:

$$0 = w_1 f_1 + w_2 f_2 + b \quad (\text{where } b = -a)$$

In three dimensions, a plane:

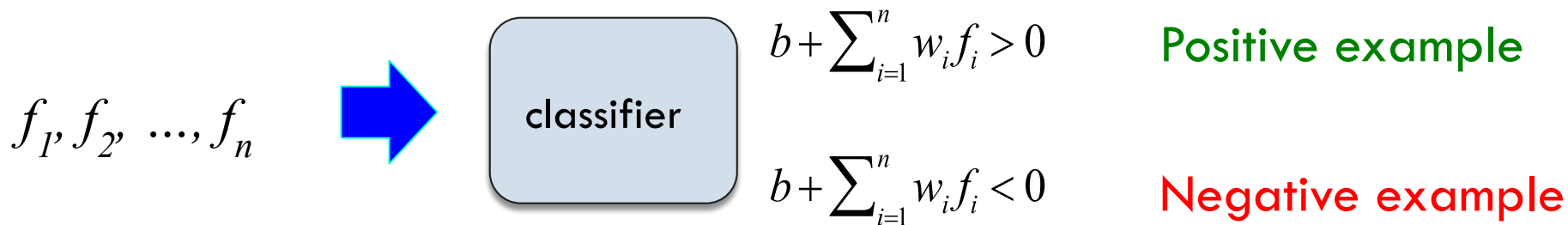
$$0 = w_1 f_1 + w_2 f_2 + w_3 f_3 + b$$

In  $n$ -dimensions, a **hyperplane**

$$0 = b + \sum_{i=1}^n w_i f_i$$

# CLASSIFYING WITH A LINEAR MODEL

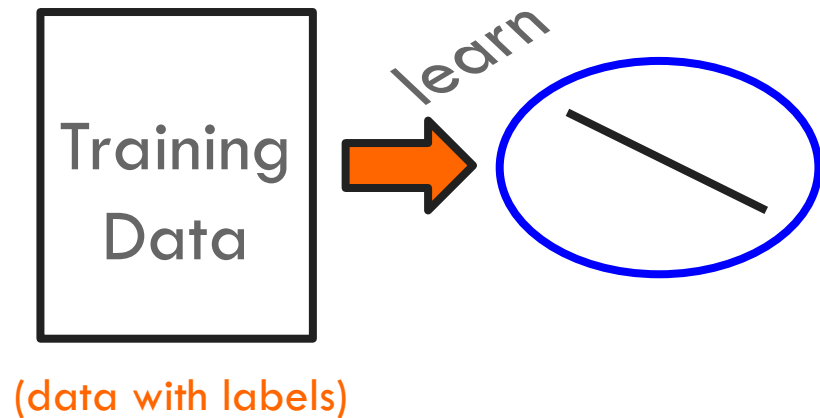
We can classify with a linear model by checking the sign:



# ONLINE LEARNING

# HOW DO WE LEARN A LINEAR MODEL?

Given a linear model (i.e. a set of weights  $w_i$  and  $b$ ) we can classify examples



How do we learn a linear model?

# LEARNING A LINEAR MODEL

Positive or negative?



# LEARNING A LINEAR MODEL

Positive or negative?



# LEARNING A LINEAR MODEL

Positive or negative?



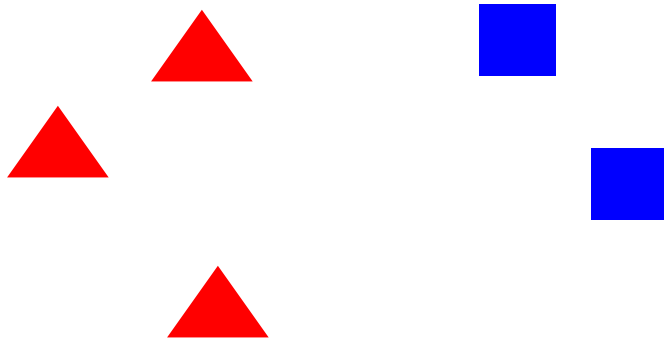
# LEARNING A LINEAR MODEL

Positive or negative?



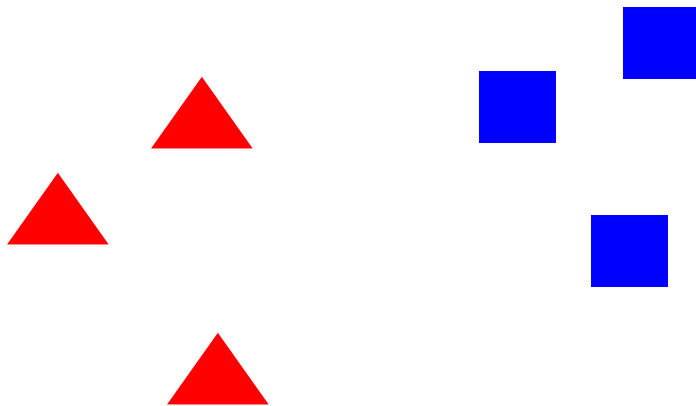
# LEARNING A LINEAR MODEL

Positive or negative?



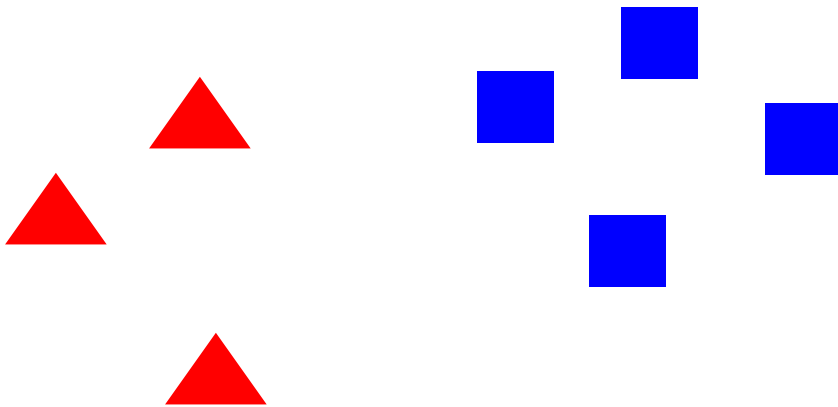
# LEARNING A LINEAR MODEL

Positive or negative?



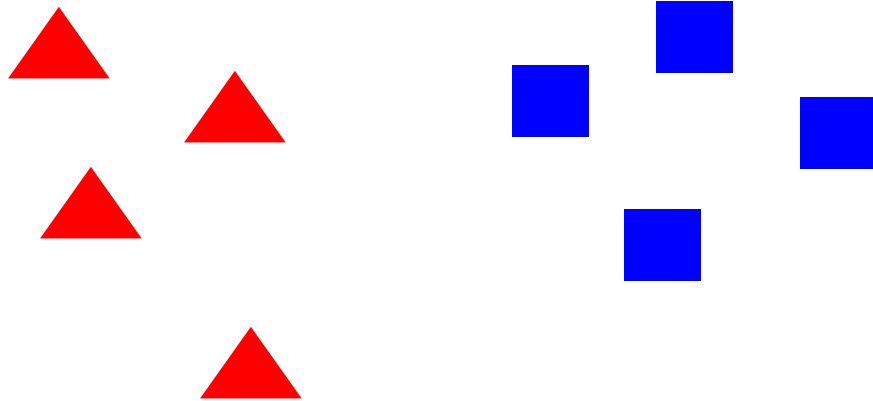
# LEARNING A LINEAR MODEL

Positive or negative?



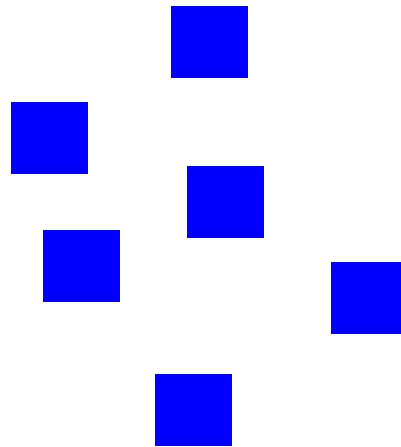
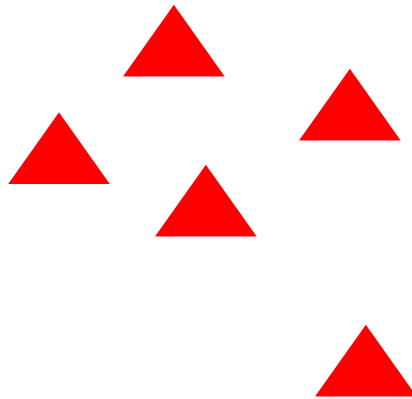
# LEARNING A LINEAR MODEL

Positive or negative?



# LEARNING A LINEAR MODEL

Positive or negative?



# LEARNING A LINEAR MODEL

How is this learning setup different than from what we have seen before?

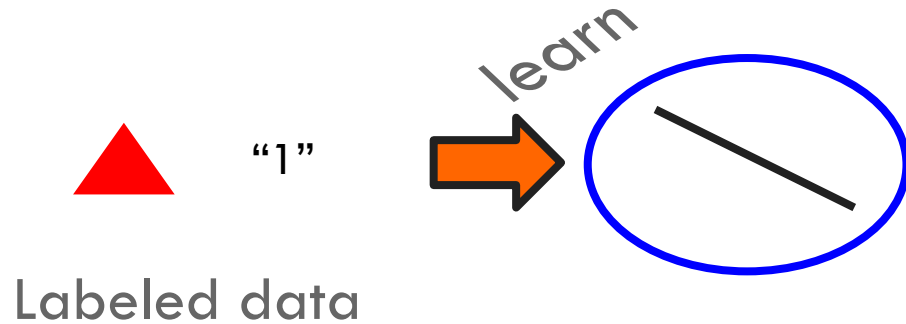
# LEARNING A LINEAR MODEL

How is this learning setup different than from what we have seen before?

**Online learning!**

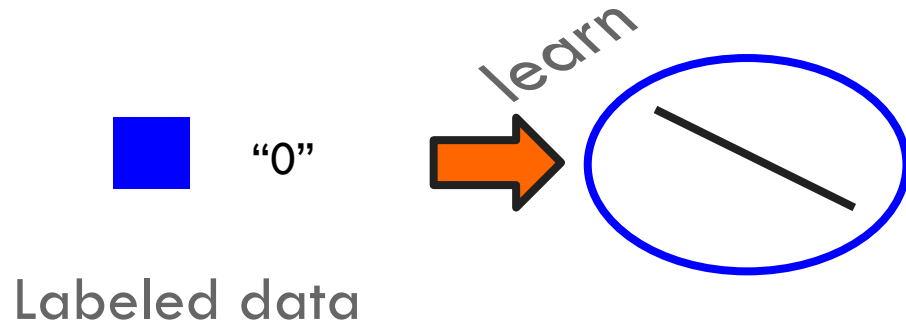
# ONLINE LEARNING ALGORITHM

We only see one example at the time!



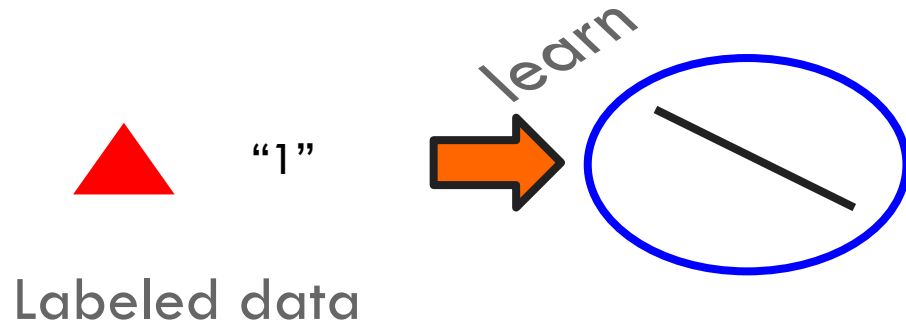
# ONLINE LEARNING ALGORITHM

We only see one example at the time!



# ONLINE LEARNING ALGORITHM

We only see one example at the time!



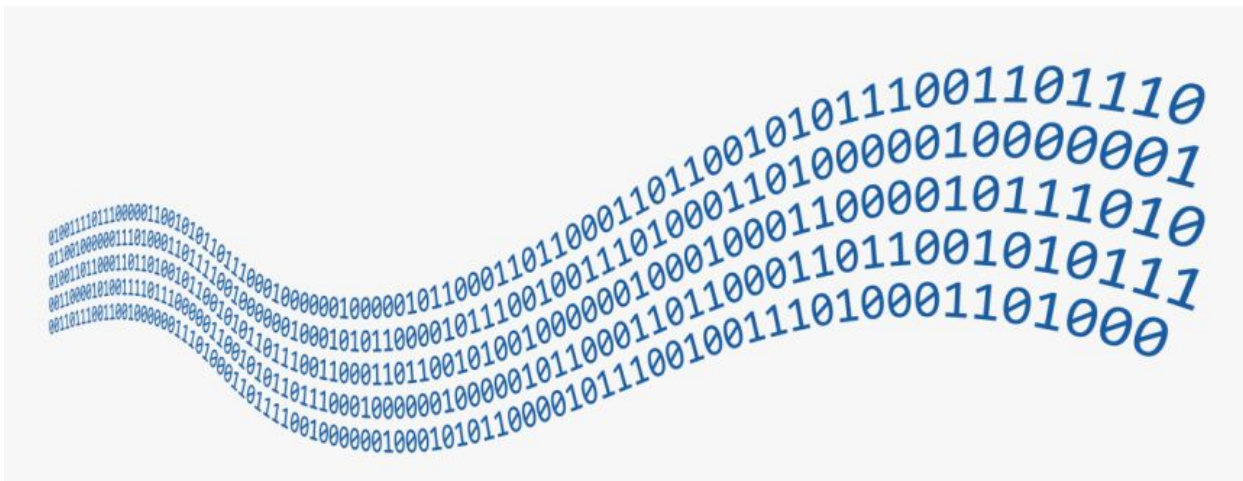
# LEARNING A LINEAR MODEL

When we need online learning?

# LEARNING A LINEAR MODEL

When we need online learning?

**Data Streams!**



# LEARNING A LINEAR MODEL

When we need online learning?

Large-scale datasets



# LEARNING A LINEAR MODEL

When we need online learning?

Privacy-preserving applications

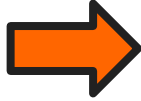


# LESSON LEARNED: ONLINE VS BATCH

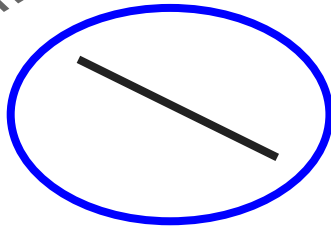
- **Batch:** Given training data  $\{(x_i, y_i) : 1 \leq i \leq n\}$ , typically i.i.d.
- **Online:** data points arrive one by one
  - The algorithm receives an unlabeled example  $x_i$
  - The algorithm predicts a classification of this example.
  - The algorithm is then told the correct answer  $y_i$ , and update its model

Training  
Data

(data with labels)



learn

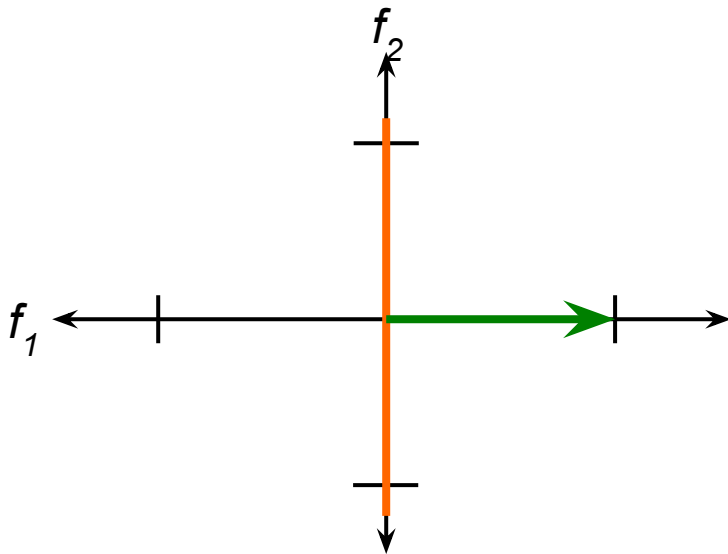


LEARNING A LINEAR MODEL

# LEARNING A LINEAR CLASSIFIER

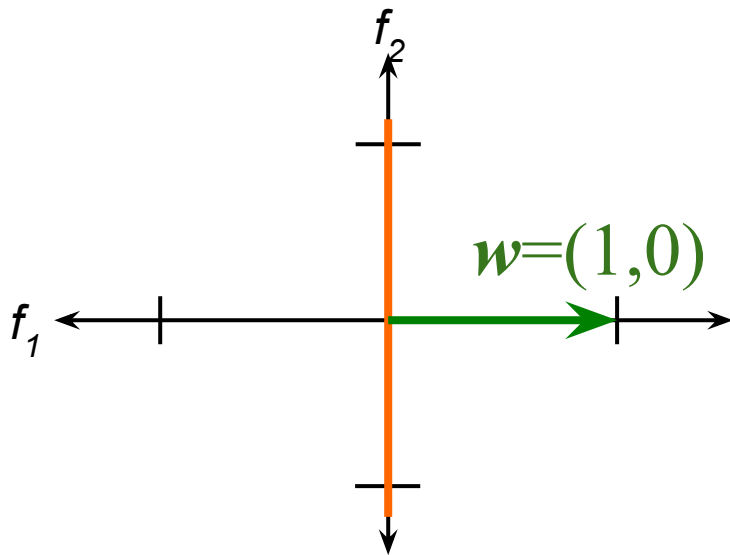
We may have:

$$w=(1,0)$$



# LEARNING A LINEAR CLASSIFIER

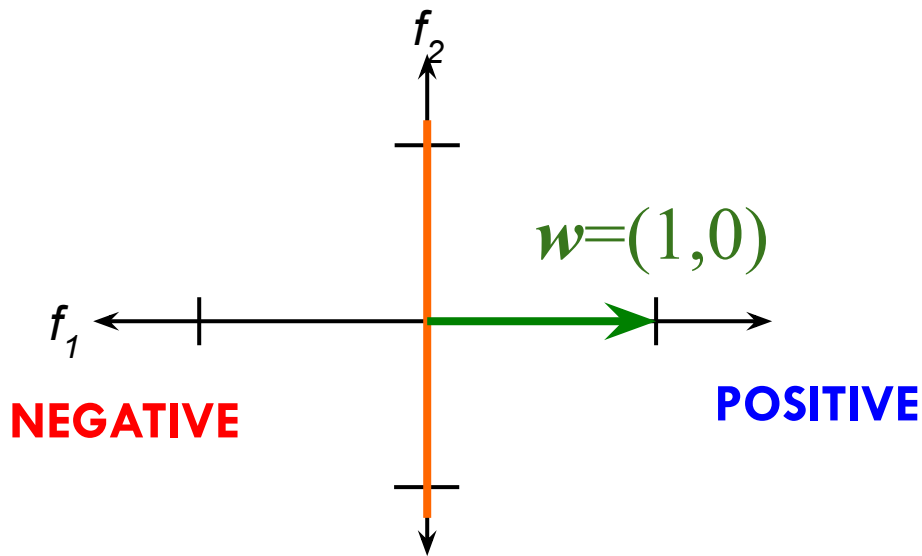
What does it mean?



# LEARNING A LINEAR CLASSIFIER

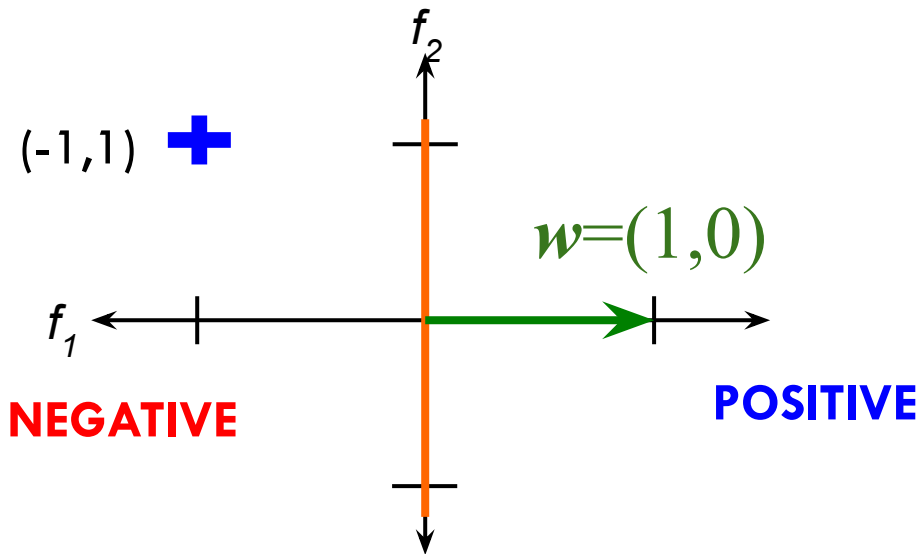
According to the rule we have seen before:

$$0 = w_1 f_1 + w_2 f_2$$



# LEARNING A LINEAR CLASSIFIER

Now a new sample arrive. It is a positive sample:



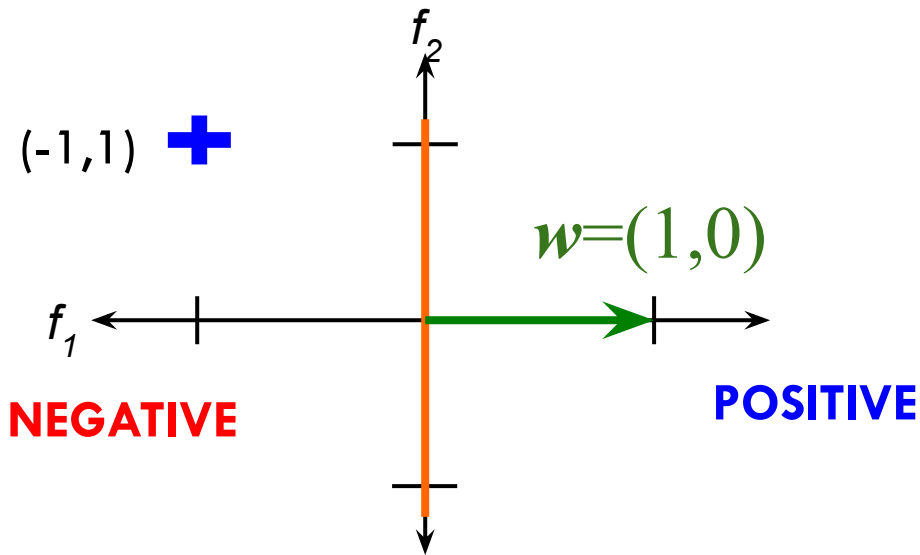
# LEARNING A LINEAR CLASSIFIER

Now a new sample arrive. It is a positive sample:

$$1 * f_1 + 0 * f_2 =$$

$$1 * -1 + 0 * 1 = -1$$

Negative, wrong!



# LEARNING A LINEAR CLASSIFIER

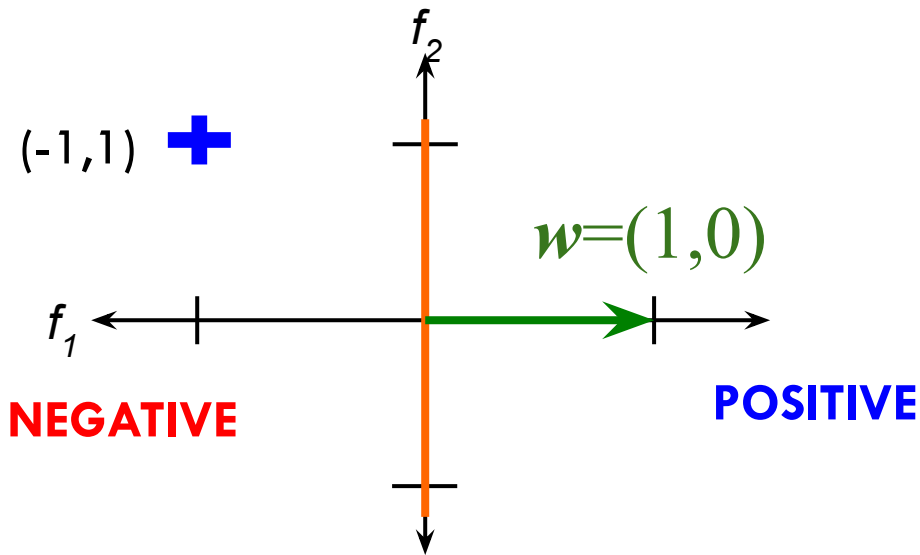
Now a new sample arrive. It is a positive sample:

$$1 * f_1 + 0 * f_2 =$$

$$1 * -1 + 0 * 1 = -1$$

Negative, wrong!

Model must be updated!



# A CLOSER LOOK AT WHY WE GOT IT WRONG

$$w_1 \quad w_2$$
$$1 * f_1 + 0 * f_2 =$$

$$\underbrace{1 * -1 + 0 * 1}_{= -1}$$

$$(-1, 1) \quad +$$

← This value should be positive!

How do we adjust  $w_1$  and  $w_2$ ?

# A CLOSER LOOK AT WHY WE GOT IT WRONG

$$w_1 \quad w_2$$
$$1 * f_1 + 0 * f_2 =$$

$$1 * -1 + 0 * 1 = -1$$

$$(-1, 1) \quad +$$

← This value should be positive!value

↑  
contributed in the  
wrong direction

←  
could have contributed  
(positive feature) but it did  
not since the weight is 0

# A CLOSER LOOK AT WHY WE GOT IT WRONG

$$w_1 \quad w_2$$
$$1 * f_1 + 0 * f_2 =$$

$$1 * -1 + 0 * 1 = -1$$

$$(-1, 1) \quad +$$

← This value should be positive!value

↑  
decrease

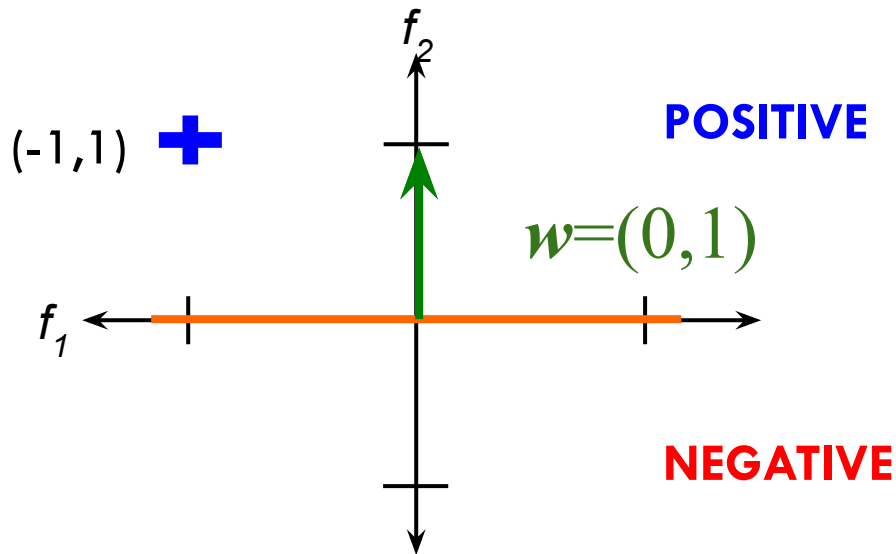
e.g. from 1 to 0

← increase

from 0 to 1

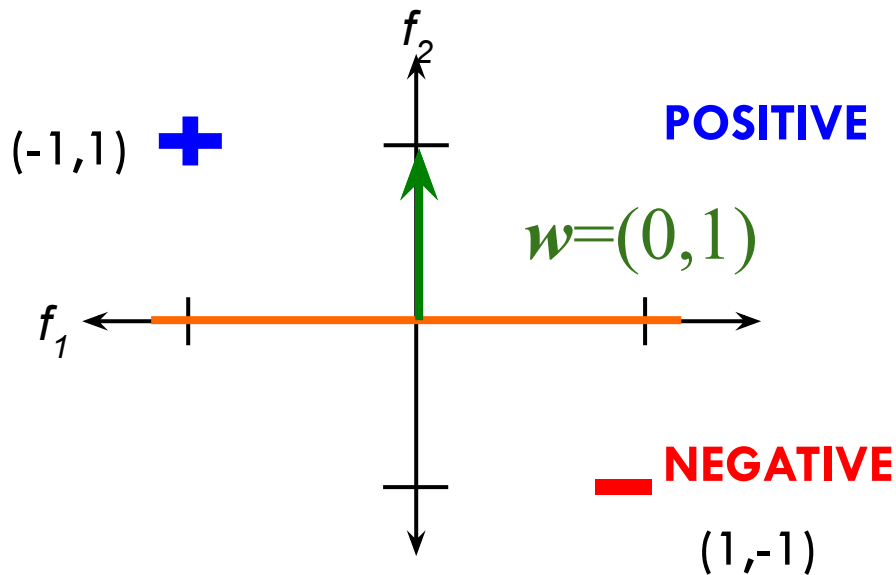
# LEARNING A LINEAR CLASSIFIER

Great! The model is successfully updated!



# LEARNING A LINEAR CLASSIFIER

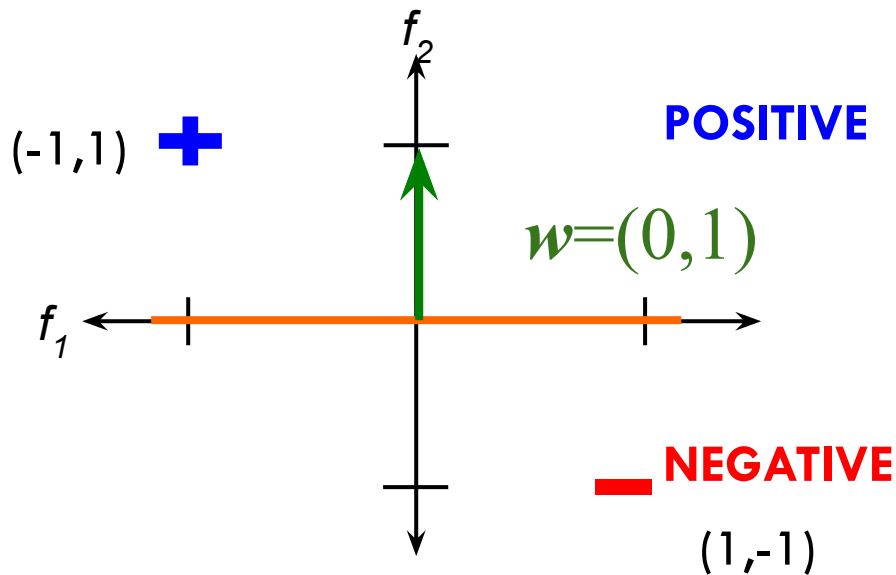
Let us continue...



# LEARNING A LINEAR CLASSIFIER

Let us continue...

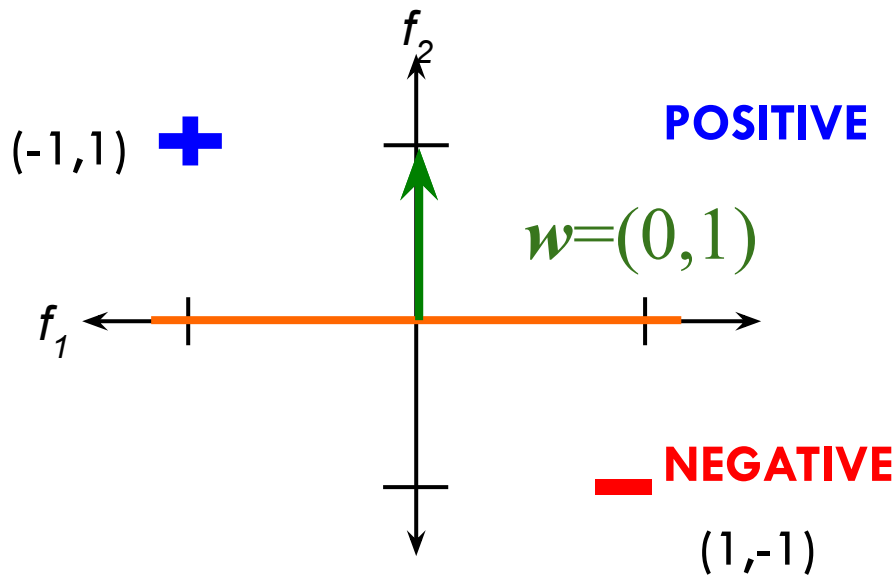
Is it correct?



# LEARNING A LINEAR CLASSIFIER

Let us continue...

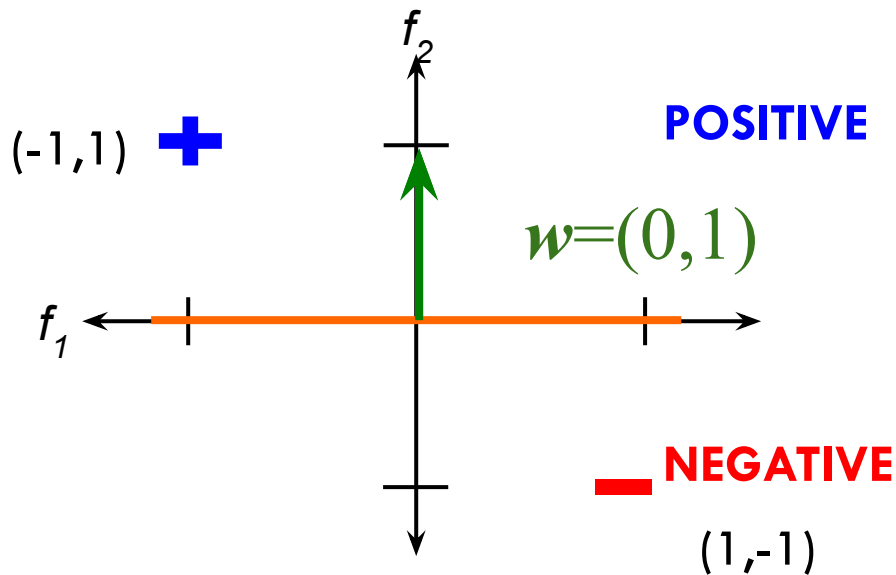
Is it correct? YES



# LEARNING A LINEAR CLASSIFIER

Let us continue...

Is it correct? YES



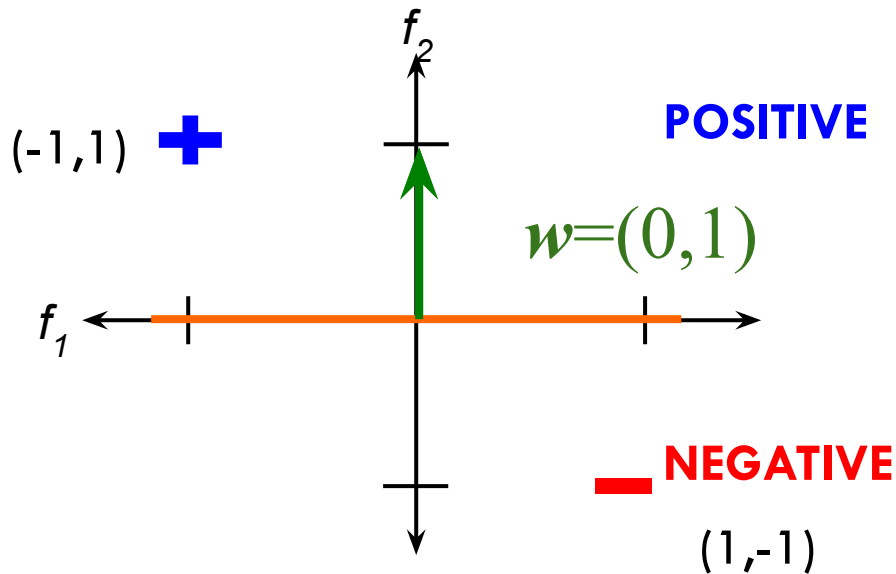
# LEARNING A LINEAR CLASSIFIER

$$0 = w_1 f_1 + w_2 f_2$$

$$0 * f_1 + 1 * f_2 =$$

$$0 * 1 + 1 * -1 = -1$$

Is it correct? YES

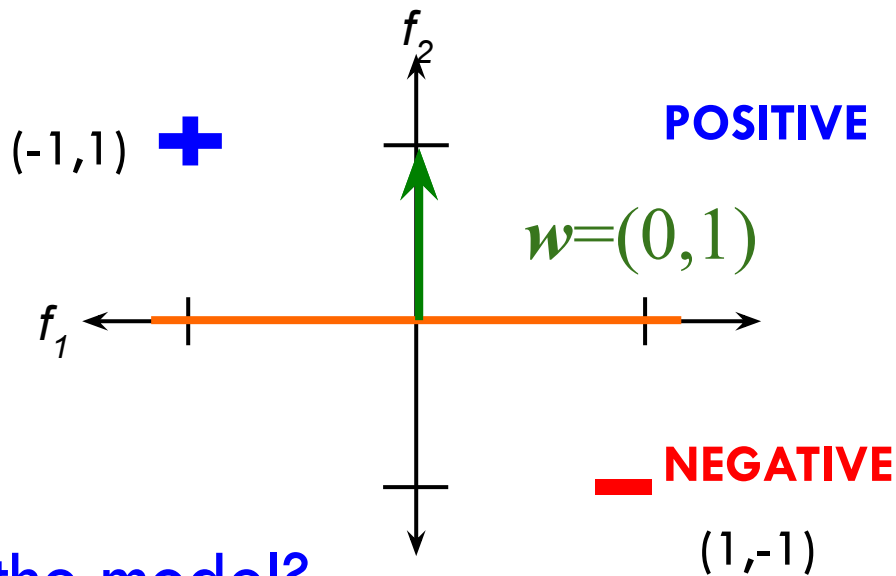


# LEARNING A LINEAR CLASSIFIER

$$0 = w_1 f_1 + w_2 f_2$$

$$0 * f_1 + 1 * f_2 =$$

$$0 * 1 + 1 * -1 = -1$$



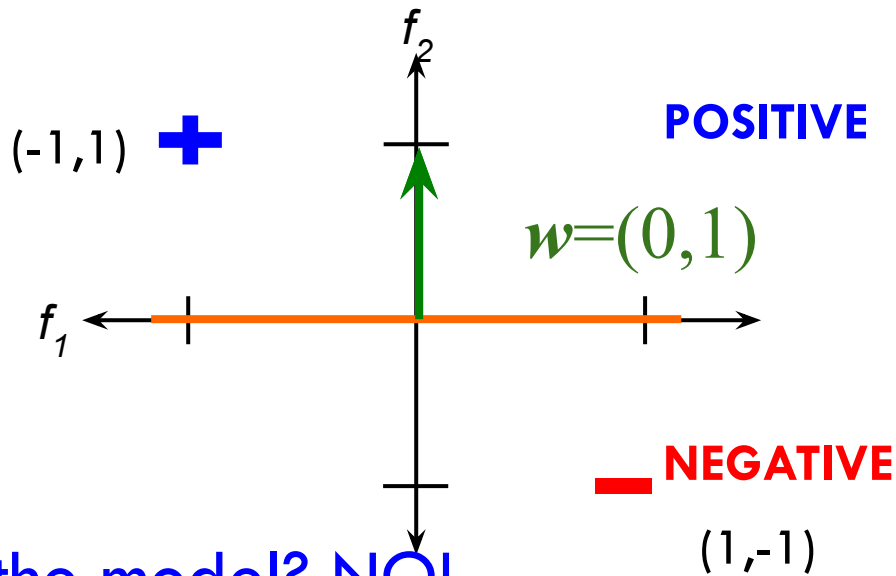
Do we need to update the model?

# LEARNING A LINEAR CLASSIFIER

$$0 = w_1 f_1 + w_2 f_2$$

$$0 * f_1 + 1 * f_2 =$$

$$0 * 1 + 1 * -1 = -1$$



Do we need to update the model? **NO!**

# LEARNING A LINEAR CLASSIFIER

Can we derive an algorithm?

# PERCEPTRON LEARNING ALGORITHM

**repeat** until convergence (or for some # of iterations):

**for** each training example  $(f_1, f_2, \dots, f_n, \text{label})$ :

check if it is correct based on the current model

**if** not correct, update all the weights:

**for** each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

# PERCEPTRON LEARNING ALGORITHM

**repeat** until convergence (or for some # of iterations):

**for** each training example  $(f_1, f_2, \dots, f_n, \text{label})$ :

label is -1/1

check if it is correct based on the current model

**if** not correct, update all the weights:

**for** each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

# PERCEPTRON LEARNING ALGORITHM

**repeat** until convergence (or for some # of iterations):

**for** each training example  $(f_1, f_2, \dots, f_n, \text{label})$ :

check if it is correct based on the current model

**if** not correct, update all the weights:

**for** each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

# PERCEPTRON LEARNING ALGORITHM

**repeat** until convergence (or for some # of iterations):

**for** each training example  $(f_1, f_2, \dots, f_n, \text{label})$ :

**check if it is correct based on the current model**

**if** not correct, update all the weights:

**for** each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

# PERCEPTRON LEARNING ALGORITHM

**repeat** until convergence (or for some # of iterations):

**for** each training example ( $f_1, f_2, \dots, f_n$ , label):

$$\text{prediction} = b + \sum_{i=1}^n w_i f_i$$

**if** not correct, update all the weights:

**for** each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

# PERCEPTRON LEARNING ALGORITHM

**repeat** until convergence (or for some # of iterations):

**for** each training example  $(f_1, f_2, \dots, f_n, \text{label})$ :

$$\text{prediction} = b + \sum_{i=1}^n w_i f_i$$

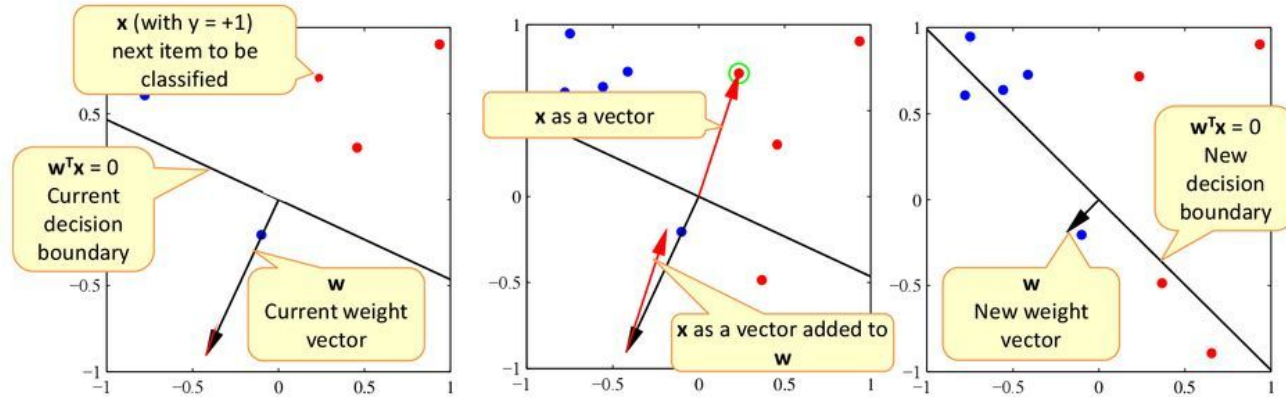
**if** *prediction is different from label*

**for** each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

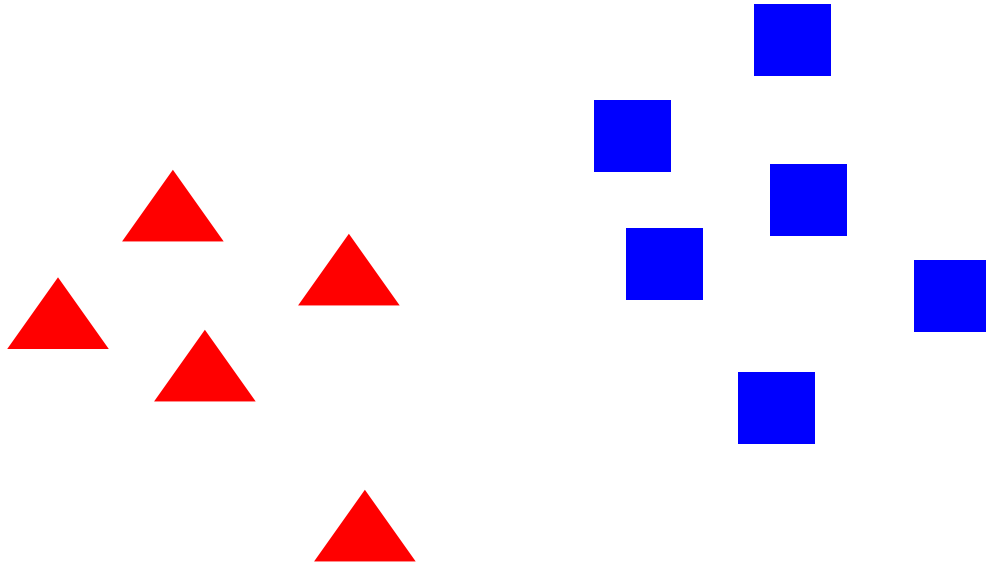
# PERCEPTRON IN ACTION



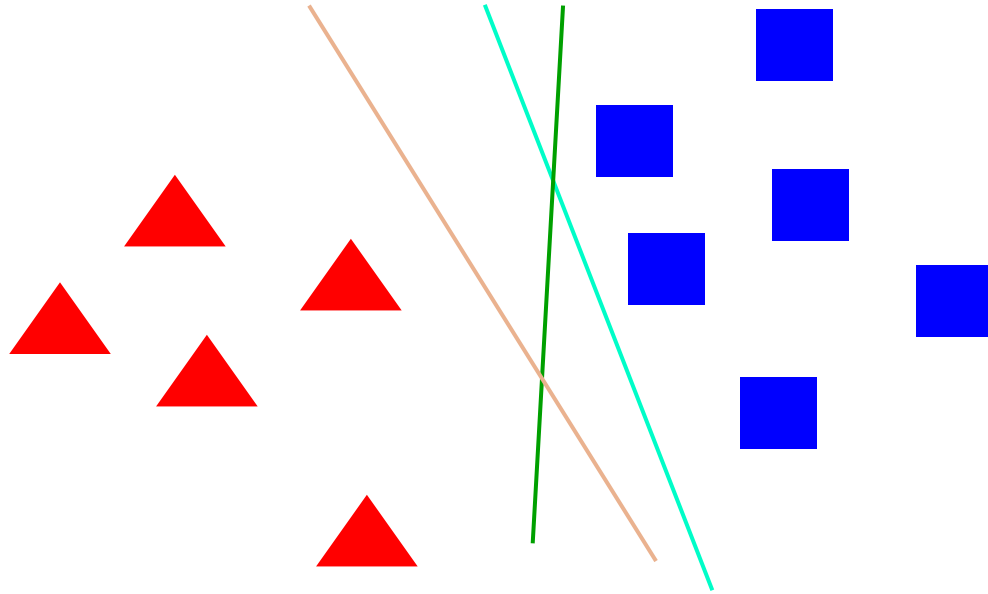
(Figures from Bishop 2006)



WHICH LINE WILL THE PERCEPTRON FIND?



# WHICH LINE WILL THE PERCEPTRON FIND?



Only guaranteed to find **some** line that separates the data!

# CONVERGENCE?

repeat until convergence (or for some # of iterations):

for each training example ( $f_1, f_2, \dots, f_n$ , label):

check if it is correct based on the current model

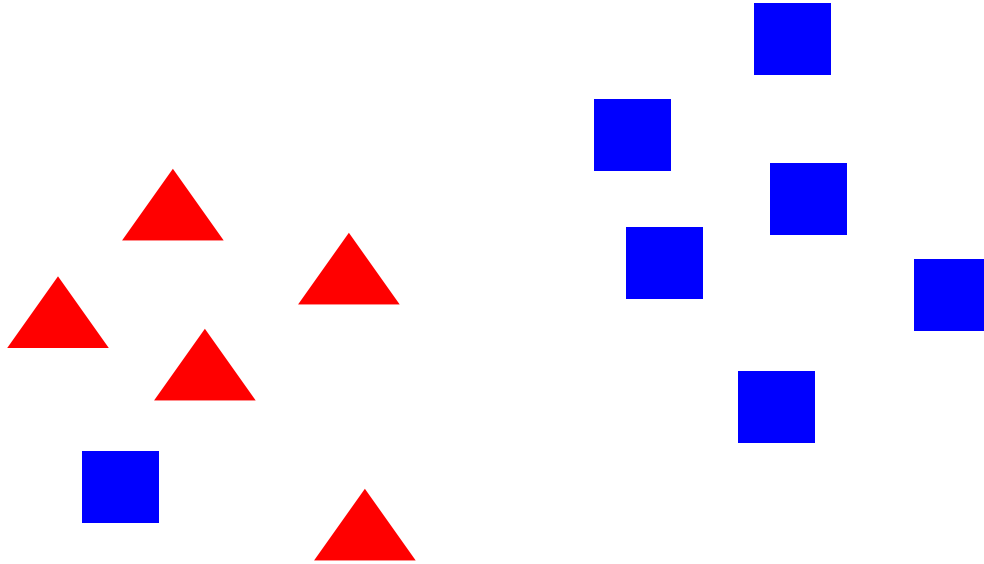
if not correct, update all the weights:

for each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

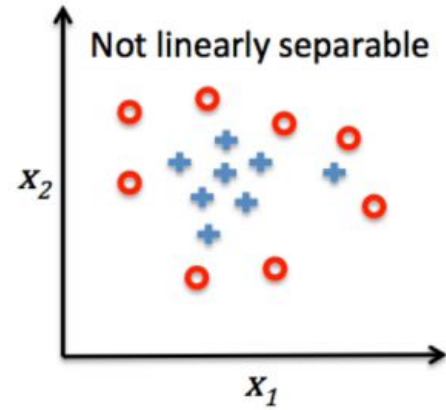
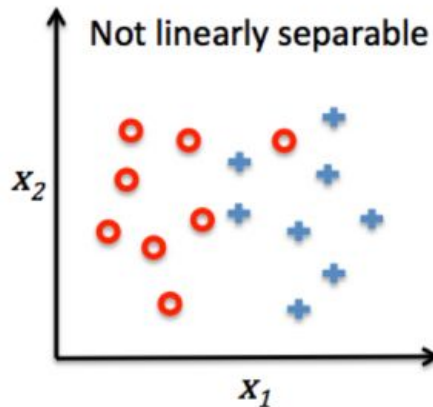
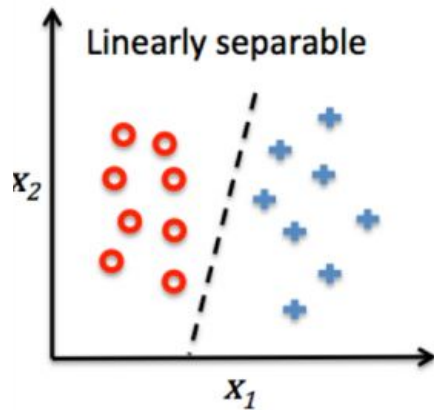
# NON SEPARABLE DATA



There will be no convergence here!

# LINEAR SEPARABLE SETS

The training instances are linearly separable if there exists a hyperplane that will separate the two classes.



# NUMBER OF ITERATIONS

repeat until convergence (or for some # of iterations):

for each training example  $(f_1, f_2, \dots, f_n, \text{label})$ :

check if it is correct based on the current model

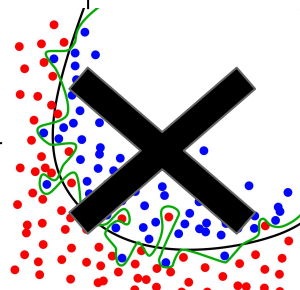
if not correct, update all the weights:

for each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

Limit overfitting!!!!



# SAMPLE ORDER?

repeat until convergence (or for some # of iterations):

for each training example  $(f_1, f_2, \dots, f_n, \text{label})$ :

check if it is correct based on the current model

if not correct, update all the weights:

for each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

# SAMPLE ORDER?

repeat until convergence (or for some # of iterations):

random sample one example ( $f_1, f_2, \dots, f_n$ , label):

check if it is correct based on the current model

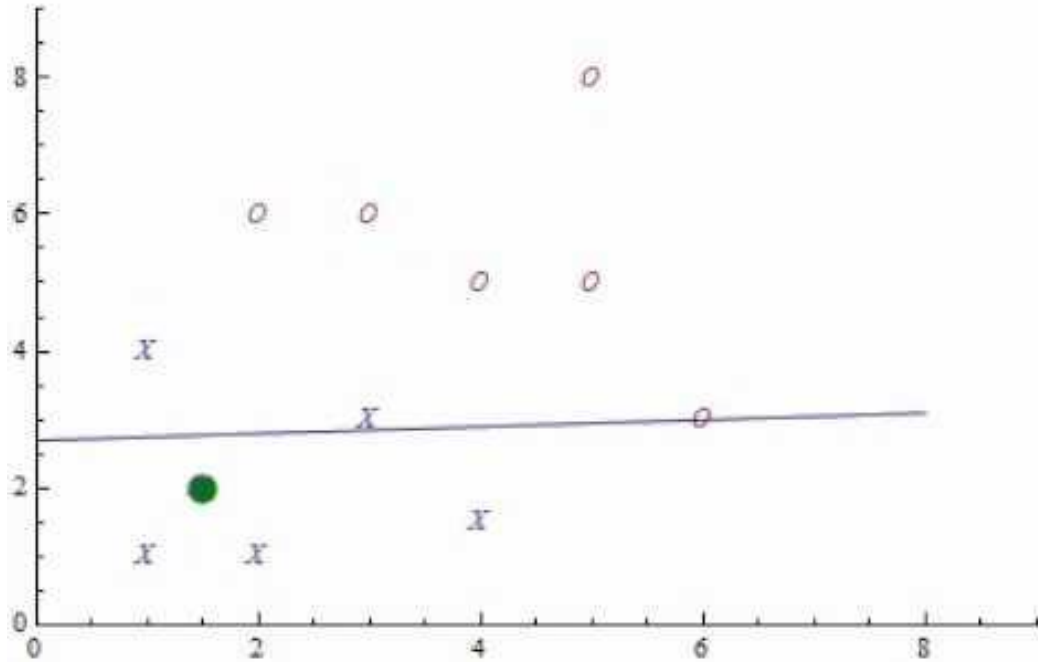
if not correct, update all the weights:

for each  $w_i$ :

$$w_i = w_i + f_i * \text{label}$$

$$b = b + \text{label}$$

# PERCEPTRON IN ACTION



# QUESTIONS?

